학사관리시스템 데이터베이스 설계 및 SQL 실습 보고서

항목	내용
과정	SKALA – Full-stack Engineering (스마트 데이터 이해 및 활용)
작성자	김지호
실습 환경	macOS / PostgreSQL 17.10 (Homebrew) / DBeaver 26.1.3
데이터베이스·스키마	`simonjiho_db.academic`
제출일	2026-07-22

이 보고서는 강의자료 「8. 종합실습 – 1」의 실습 단계와 제출 기준에 맞추어 다음을 수록한다.

1. 학사관리시스템 요구사항과 설계 방향
2. PostgreSQL 접속 결과 화면
3. 문항별 실습 결과 (SQL문 + 실행 결과 화면)

1. 실습 목적

PostgreSQL 환경을 구성·연결하고 학사관리시스템 ERD를 설계한 뒤, 제약조건을 포함한 DDL로 테이블을 생성한다. 각 테이블에 최소 10건 이상의 샘플 데이터를 입력하고 `WHERE`·`ORDER BY` 기초 조회, `COALESCE`·`CASE WHEN`·날짜 함수 활용 조회, 수강신청 교차 테이블 `JOIN` 조회를 수행한다.

2. 요구사항과 설계 방향

2.1 업무 요구사항 요약

1. 학과의 코드와 명칭을 중복 없이 관리한다.
2. 학생의 학번, 이름, 이메일, 학년(1~4), 학적 상태, 입학일, 전화번호(선택) 및 소속 학과를 관리한다.
3. 교수의 교번, 이름, 이메일, 재직 상태 및 소속 학과를 관리한다.
4. 교과목 코드, 교과목명 및 학점(1~3)을 관리한다.
5. 교과목을 연도·학기·분반별 강좌로 개설하고 담당 교수, 정원, 개설 상태를 지정한다. 동일 교과목·연도·학기·분반 조합은 중복될 수 없다.
6. 학생과 강좌는 N:M 관계이므로 수강신청 교차 테이블로 해소한다.
7. 동일 학생의 동일 강좌 중복 신청을 금지한다.
8. 수강 상태(신청 완료/취소), 신청·취소 시각과 최종 성적(A+~F)을 관리한다. 취소 기록은 삭제하지 않고 상태와 취소 시각으로 남긴다.

2.2 설계 절차

강의자료의 ERD 작성 순서에 따라 요구사항 수집 → 엔티티 도출 → 속성 정의 → 정규화·비정규화 검토 → 관계 설정 → 카디널리티 명시 → 도구 시각화(dbdiagram.io) → 최종 검토 순으로 진행했다.

- **정규화**: 학사 영역 6개 엔티티는 1NF·2NF·3NF를 만족함을 확인했다. 학과명·교과목명 등 다른 엔티티에서 얻을 수 있는 설명값은 중복 저장하지 않는다.
- **파생값 분리**: `미등록`, 학년 안내 문구, `미정`, 수강 인원, 잔여석, 입학 연도는 저장하지 않고 조회 시 계산한다.
- **영역 구분**: 전체 설계에는 현재 학사정보(`academic`)와 변경이력·감사(`audit`) 두 스키마를 두었다. 이번 문항 실습의 구현·조회 범위는 `academic` 스키마의 핵심 6개 테이블(학과, 학생, 교수, 교과목, 개설강좌, 수강신청)이다.

2.3 설계 방향

- 반복되는 학과, 교수, 교과목 정보를 별도 테이블로 분리하여 중복과 갱신 이상을 줄였다.
- 모든 테이블에 대리 기본키(`BIGINT IDENTITY`)를 사용하고 학번·교번·학과 코드·교과목 코드 같은 업무 식별자에는 `UNIQUE`를 적용했다.
- 학년, 학점, 연도, 학기, 정원에는 `CHECK`를 적용하고 상태 및 성적에는 PostgreSQL `ENUM`을 사용했다.
- 외래키에는 `ON DELETE RESTRICT`, `ON UPDATE RESTRICT`를 적용하여 참조 중인 학사정보가 실수로 삭제되지 않게 했다.
- 성적 미확정과 전화번호 미등록은 `NULL`로 저장하고 조회할 때 `미정`, `미등록` 문구로 변환한다.

3. PostgreSQL 접속 및 환경 확인

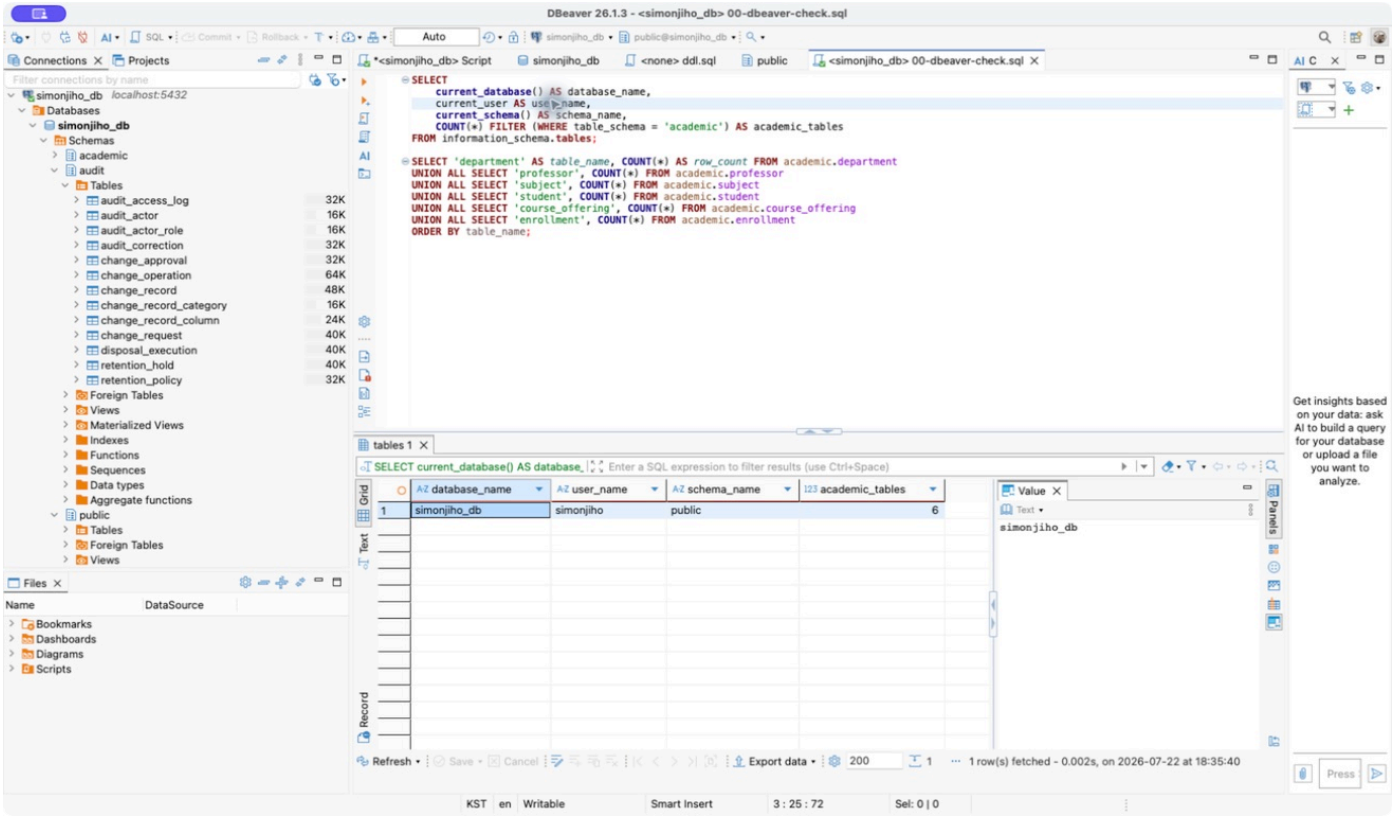
DBeaver에서 `simonjiho_db`(localhost:5432)에 접속한 뒤 아래 SQL로 접속 정보와 `academic` 스키마 구성을 확인하였다.

실행 SQL

```
SELECT
    current_database() AS database_name,
    current_user AS user_name,
    current_schema() AS schema_name,
    COUNT(*) FILTER (WHERE table_schema = 'academic') AS academic_tables
FROM information_schema.tables;
```

실행 결과 화면

데이터베이스 `simonjiho_db`, 사용자 `simonjiho`로 접속되었고 `academic` 스키마에 테이블 6개가 구성되어 있음을 확인하였다.
(`current_schema()`가 `public`으로 표시된 것은 DBeaver 기본 search_path 때문이며, 실습 대상 테이블은 모두 `academic` 스키마에 있다.)



4. DDL - 스키마와 테이블 생성

`CREATE SCHEMA academic` 실행 후 학과, 학생, 교수, 교과목, 개설강좌, 수강신청의 6개 테이블을 PK, FK, UK, NOT NULL, CHECK, ENUM 제약조건과 함께 생성하였다. psql로 처음부터 재현할 수 있는 전체 스크립트(`CREATE DATABASE` 포함)는 `sql/01-schema.sql`에 수록하였다.

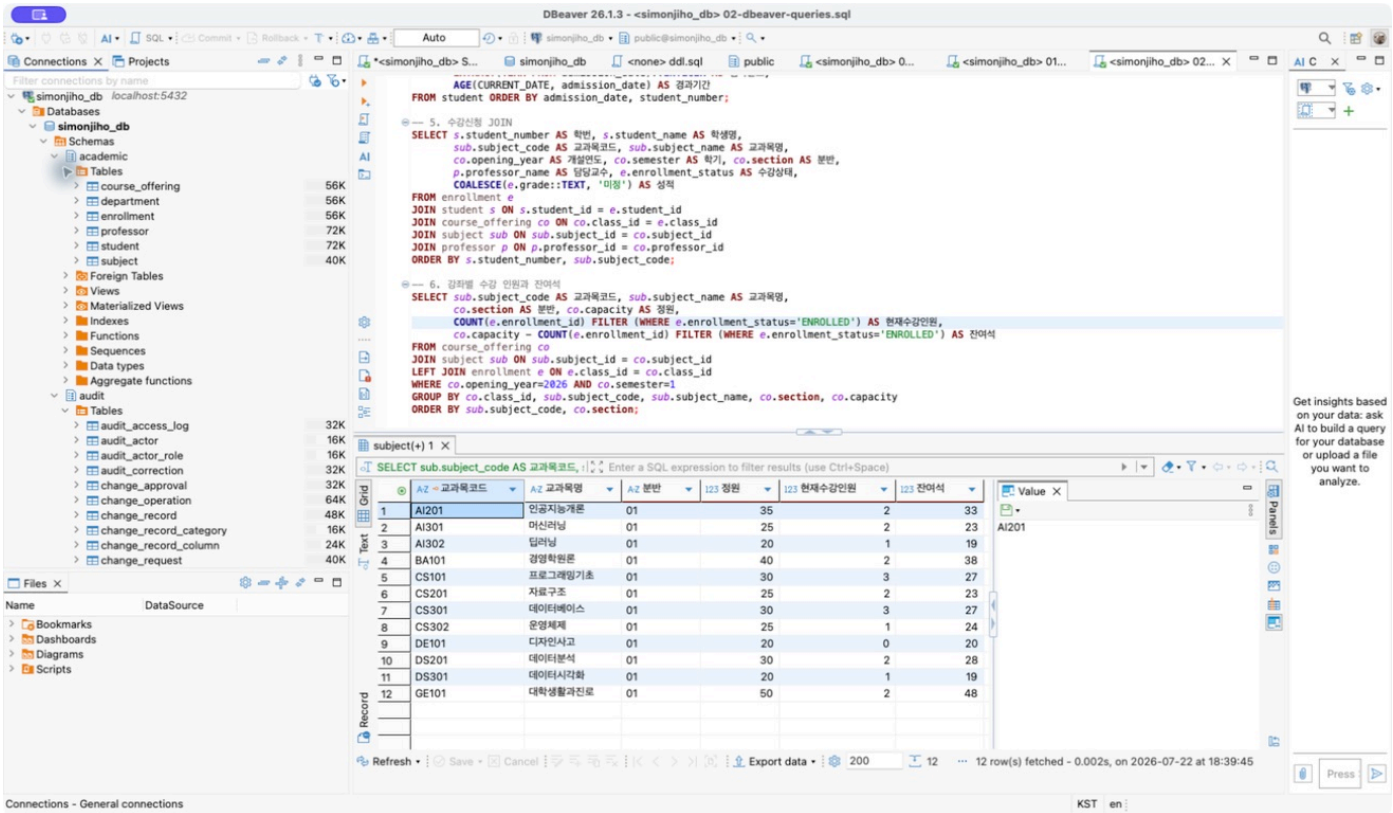
핵심 DDL 예시 — 수강신청 교차 테이블

```
CREATE TABLE enrollment (
    enrollment_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    student_id BIGINT NOT NULL REFERENCES student(student_id),
    class_id BIGINT NOT NULL REFERENCES course_offering(class_id),
    enrollment_status enrollment_status NOT NULL DEFAULT 'ENROLLED',
    enrolled_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    cancelled_at TIMESTAMPTZ,
    grade grade_code,
    UNIQUE (student_id, class_id),
    CHECK (
        (enrollment_status = 'ENROLLED' AND cancelled_at IS NULL)
        OR (enrollment_status = 'CANCELLED' AND cancelled_at IS NOT NULL)
    )
);
```

- `UNIQUE (student_id, class_id)` : 동일 학생의 동일 강좌 중복 신청 금지
- `CHECK` : 신청 완료 상태에는 취소 시각이 없고, 취소 상태에는 취소 시각이 반드시 존재

실행 결과 화면

DBeaver 네비게이터에서 `academic` 스키마에 6개 테이블(`course_offering`, `department`, `enrollment`, `professor`, `student`, `subject`)이 생성된 것을 확인하였다.



5. DML - 샘플 데이터 입력

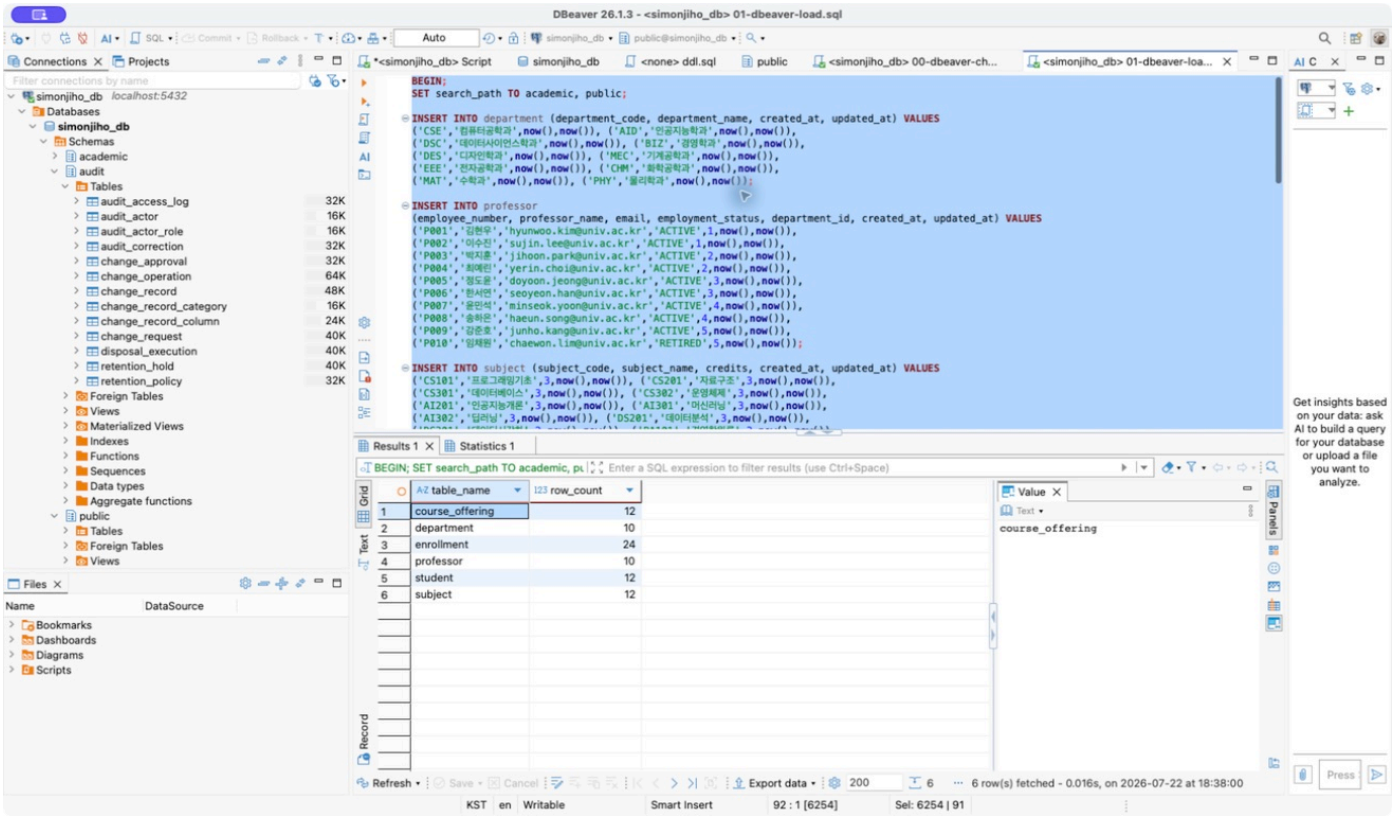
학과 10건, 교수 10건, 교과목 12건, 학생 12건, 개설강좌 12건, 수강신청 24건을 입력하여 모든 테이블을 최소 10건 이상으로 구성하였다. 수강신청에는 신청 완료 21건과 취소 3건, 성적 확정 17건과 미확정 사례를 함께 포함하였다.

DBeaver 캡처 환경에 사용한 입력 SQL은 `sql/01-dbeaver-load.sql`, psql 재현용 입력 SQL은 `sql/02-sample-data.sql`에 수록하였다.

입력 건수 확인 SQL

```
SELECT 'department' AS table_name, COUNT(*) AS row_count FROM department
UNION ALL SELECT 'professor', COUNT(*) FROM professor
UNION ALL SELECT 'subject', COUNT(*) FROM subject
UNION ALL SELECT 'student', COUNT(*) FROM student
UNION ALL SELECT 'course_offering', COUNT(*) FROM course_offering
UNION ALL SELECT 'enrollment', COUNT(*) FROM enrollment
ORDER BY table_name;
```

실행 결과 화면

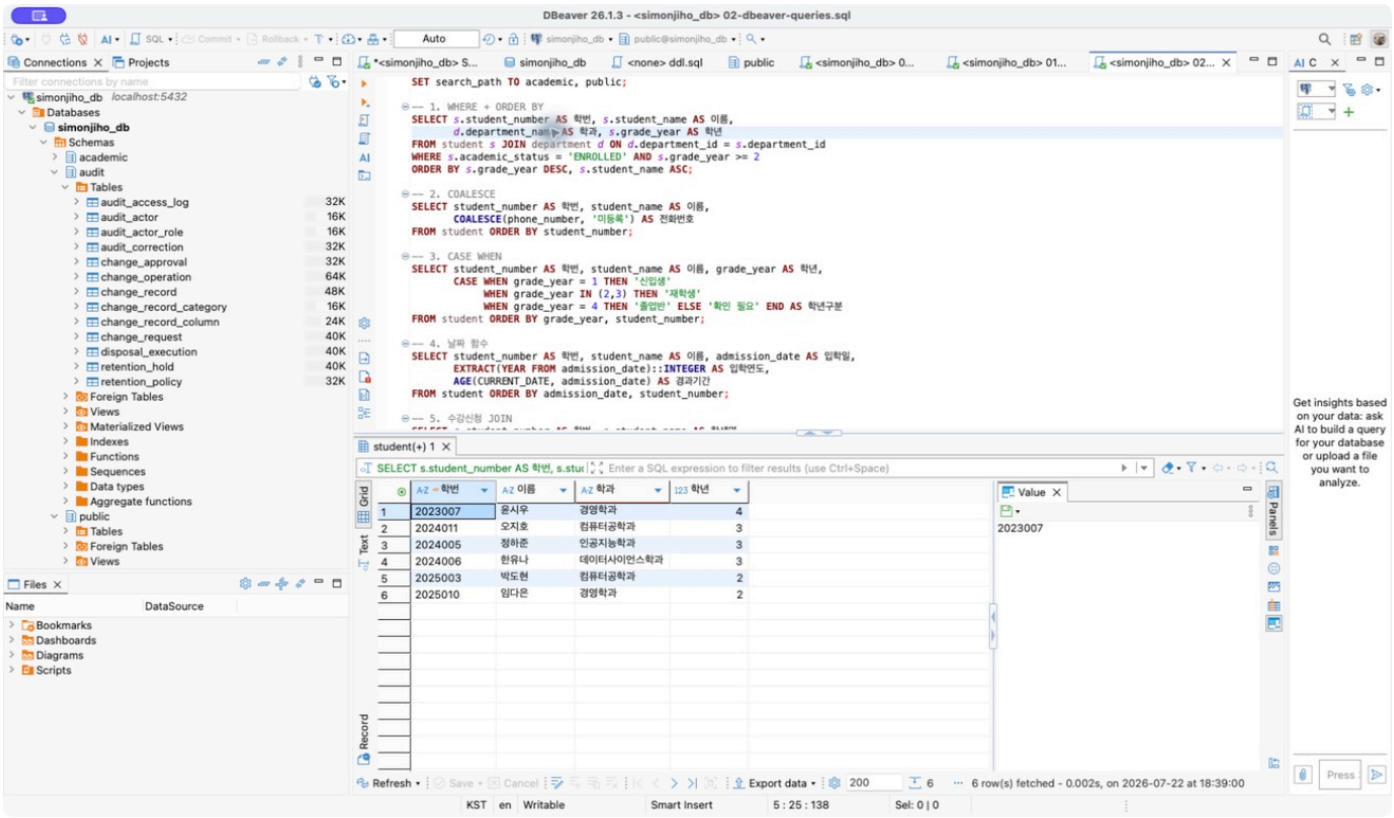


6. 문항별 실습 결과

6.1 WHERE와 ORDER BY

재학 상태이면서 2학년 이상인 학생을 학년 내림차순, 이름 오름차순으로 조회하였다. 조건에 맞는 학생 6명이 4학년부터 정렬되어 조회되었다.

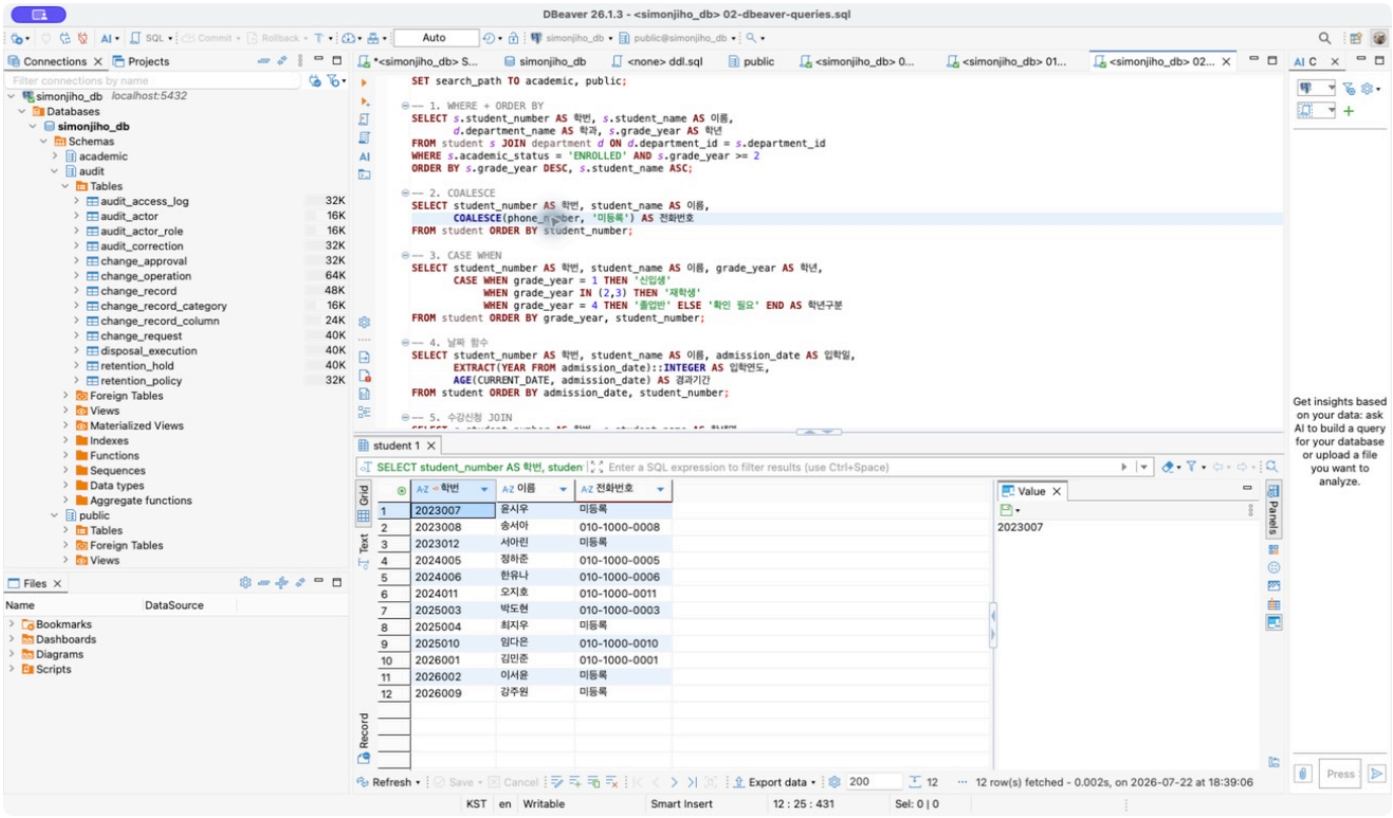
```sql
SELECT s.student_number AS 학번,
       s.student_name AS 이름,
       d.department_name AS 학과,
       s.grade_year AS 학년
FROM student s
JOIN department d ON d.department_id = s.department_id
WHERE s.academic_status = 'ENROLLED'
  AND s.grade_year >= 2
ORDER BY s.grade_year DESC, s.student_name ASC;
```



6.2 COALESCE로 NULL 처리

전화번호가 `NULL`인 학생은 `미등록`으로 표시하였다. 전체 12명 중 전화번호 미보유 학생 5명이 `미등록`으로 표시되었다.

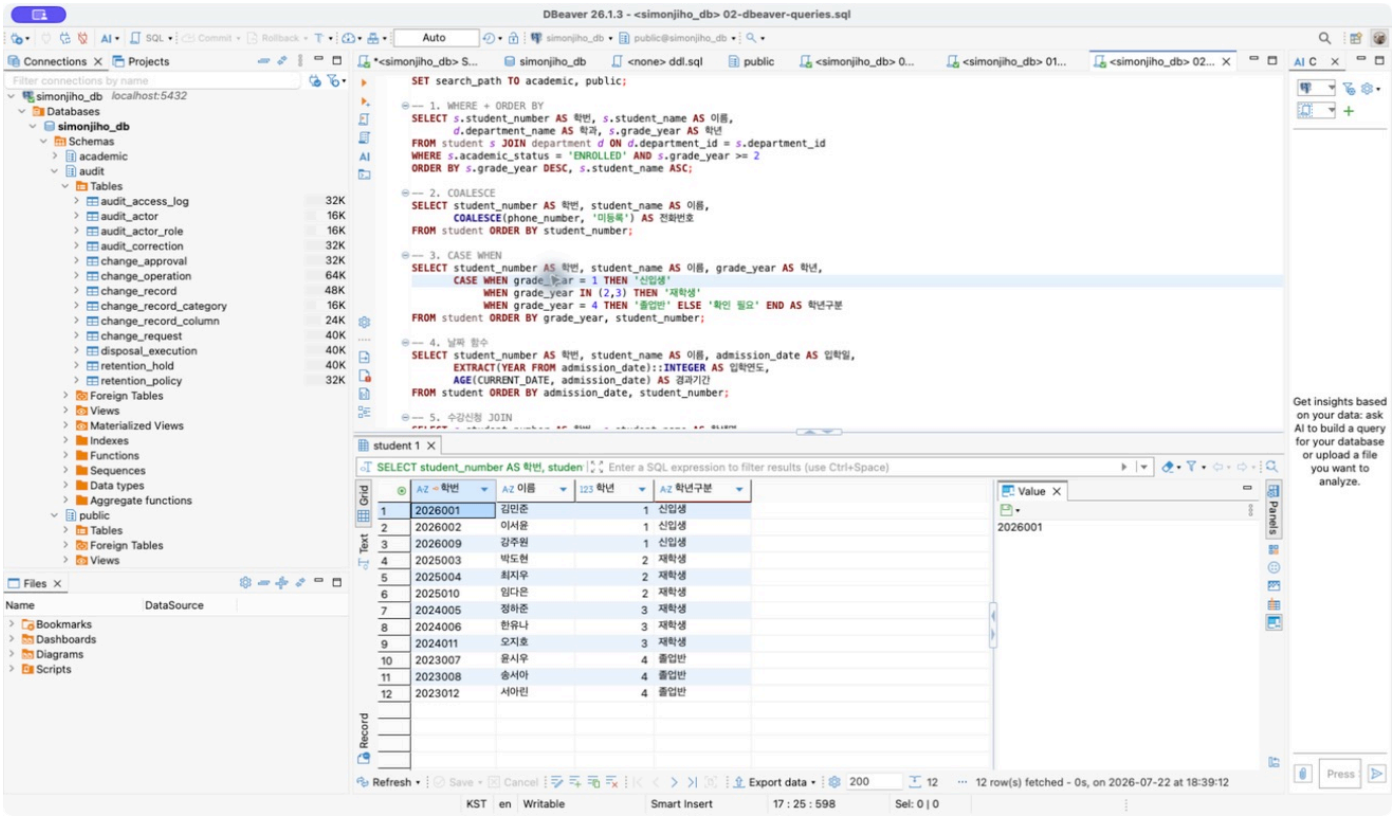
```sql
SELECT student_number AS 학번,
       student_name AS 이름,
       COALESCE(phone_number, '미등록') AS 전화번호
FROM student
ORDER BY student_number;
```



6.3 CASE WHEN으로 학년 구분

1학년은 신입생, 2·3학년은 재학생, 4학년은 졸업반으로 표시하였다. 1학년 3명, 2·3학년 6명, 4학년 3명이 각각 구분되어 표시되었다.

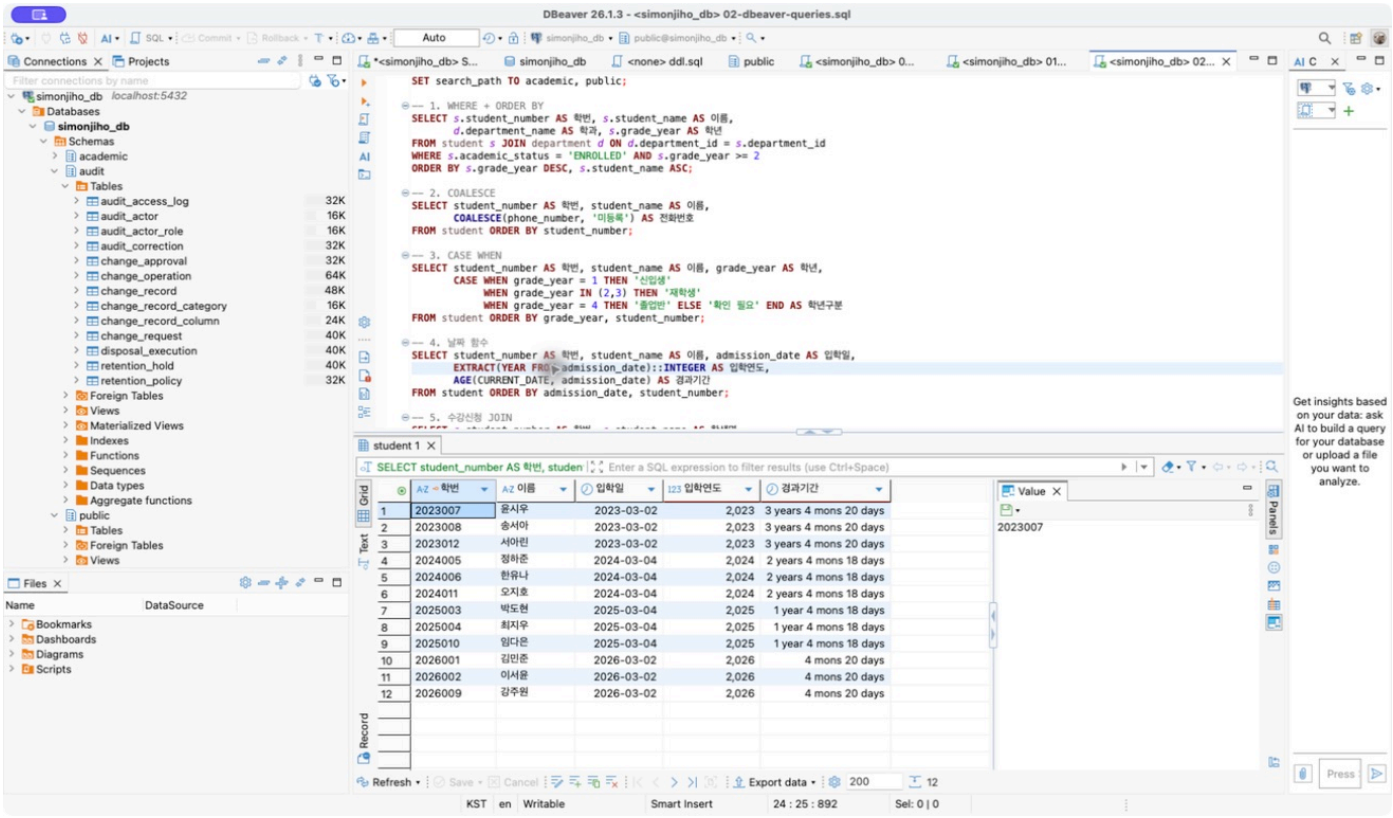
```
SELECT student_number AS 학번,
       student_name AS 이름,
       grade_year AS 학년,
       CASE
           WHEN grade_year = 1 THEN '신입생'
           WHEN grade_year IN (2, 3) THEN '재학생'
           WHEN grade_year = 4 THEN '졸업반'
           ELSE '확인 필요'
       END AS 학년구분
FROM student
ORDER BY grade_year, student_number;
```



6.4 날짜 함수 활용

`EXTRACT`로 입학일에서 입학 연도(2023~2026)를 추출하고 `AGE`로 현재 날짜 기준 경과 기간을 계산하였다.

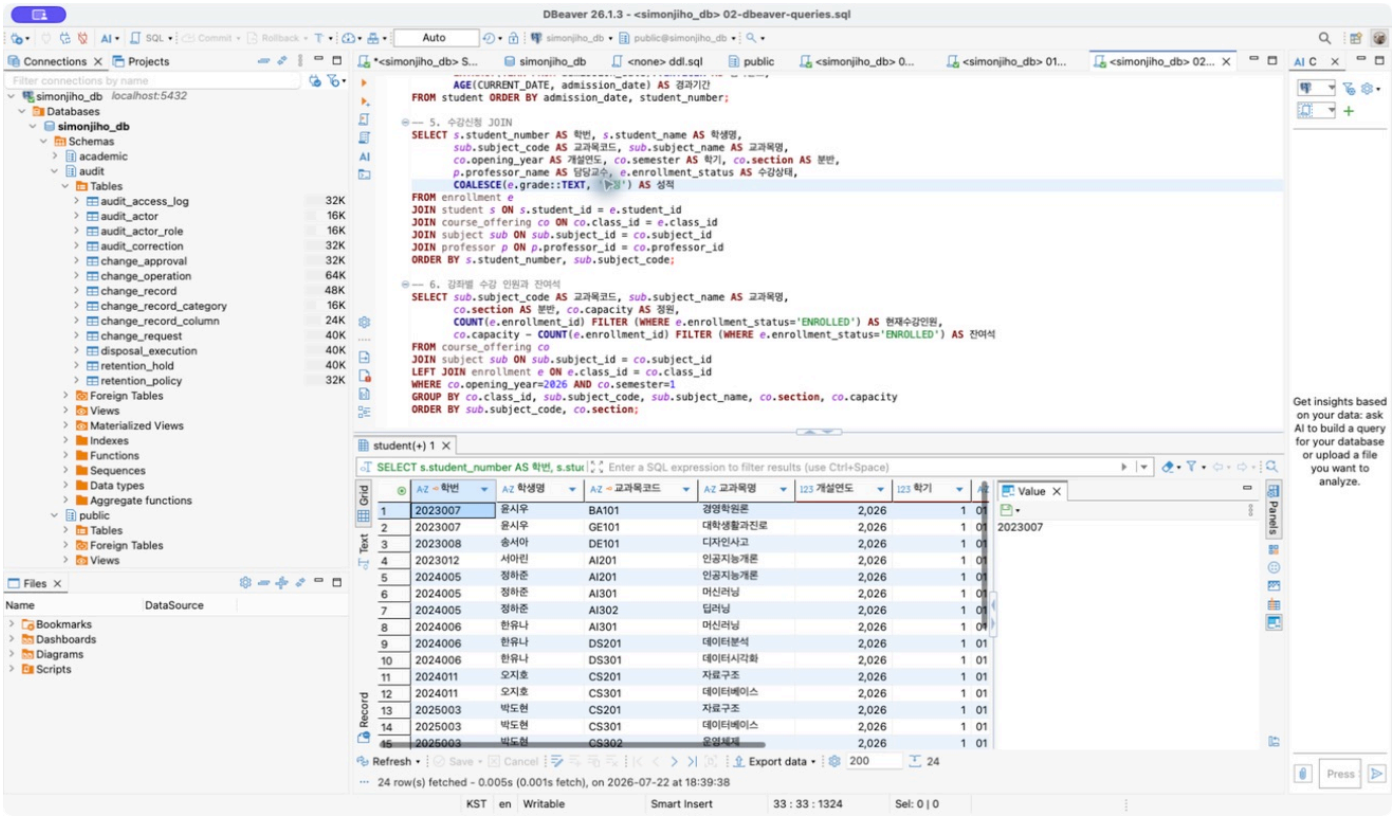
```
SELECT student_number AS 학번,
       student_name AS 이름,
       admission_date AS 입학일,
       EXTRACT(YEAR FROM admission_date)::INTEGER AS 입학연도,
       AGE(CURRENT_DATE, admission_date) AS 경과기간
FROM student
ORDER BY admission_date, student_number;
```



6.5 수강신청 교차 테이블 JOIN

수강신청을 중심으로 학생, 개설강좌, 교과목, 교수를 연결하여 학생별 수강 과목과 담당 교수, 수강 상태, 성적을 조회하였다. 전체 24건이 조회되었고 성적이 확정되지 않았거나 취소된 수강 건은 `미정`으로 표시되었다.

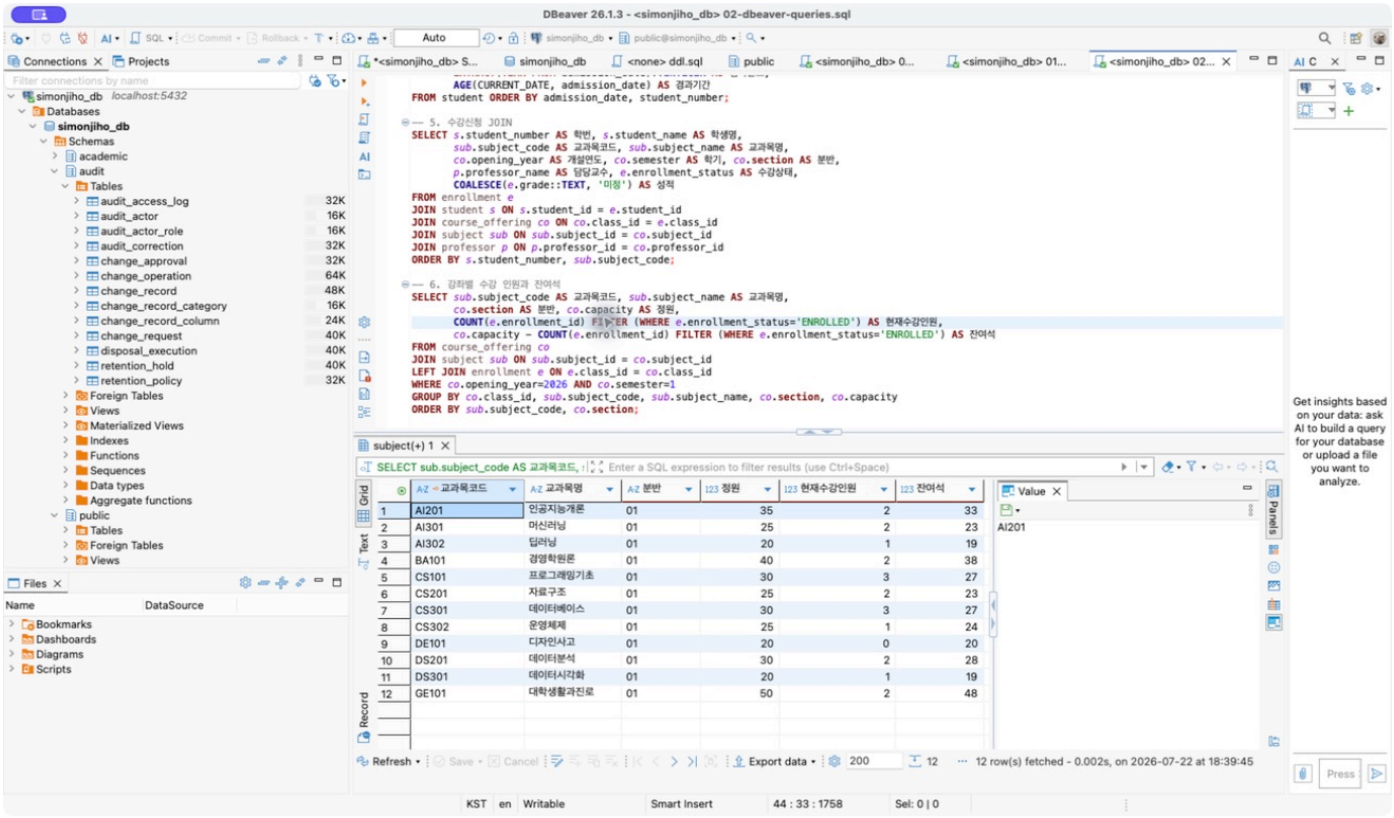
```
SELECT s.student_number AS 학번,
       s.student_name AS 학생명,
       sub.subject_code AS 교과목코드,
       sub.subject_name AS 교과목명,
       co.opening_year AS 개설연도,
       co.semester AS 학기,
       co.section AS 분반,
       p.professor_name AS 담당교수,
       e.enrollment_status AS 수강상태,
       COALESCE(e.grade::TEXT, '미정') AS 성적
FROM enrollment e
JOIN student s ON s.student_id = e.student_id
JOIN course_offering co ON co.class_id = e.class_id
JOIN subject sub ON sub.subject_id = co.subject_id
JOIN professor p ON p.professor_id = co.professor_id
ORDER BY s.student_number, sub.subject_code;
```



6.6 강좌별 수강 인원과 잔여석

조건부 집계(`FILTER`)로 신청 완료 건만 수강 인원에 포함하고, 정원에서 현재 수강 인원을 빼서 잔여석을 구하였다. 취소된 수강 건은 인원 집계에서 제외된다.

```
SELECT sub.subject_code AS 교과목코드,
       sub.subject_name AS 교과목명,
       co.section AS 분반,
       co.capacity AS 정원,
       COUNT(e.enrollment_id) FILTER (
           WHERE e.enrollment_status = 'ENROLLED'
       ) AS 현재수강인원,
       co.capacity - COUNT(e.enrollment_id) FILTER (
           WHERE e.enrollment_status = 'ENROLLED'
       ) AS 잔여석
FROM course_offering co
JOIN subject sub ON sub.subject_id = co.subject_id
LEFT JOIN enrollment e ON e.class_id = co.class_id
WHERE co.opening_year = 2026
  AND co.semester = 1
GROUP BY co.class_id, sub.subject_code, sub.subject_name,
         co.section, co.capacity
ORDER BY sub.subject_code, co.section;
```



7. psql 일괄 재현 검증

DBeaver 실습과 별도로, 빈 환경에서 동일 결과를 재현할 수 있도록 psql 일괄 실행 스크립트를 구성하고 검증하였다.

```
\i run-all.sql
```

`run-all.sql`은 `sql/01-schema.sql`(데이터베이스·스키마·테이블 생성) → `sql/02-sample-data.sql`(샘플 데이터 입력) → `sql/03-queries.sql`(문항별 조회)을 순서대로 실행한다. PostgreSQL 17.10에서 오류 없이 실행되었으며 전체 실행 로그는 `results/validation-output.txt`에 수록하였다.

8. 실습 결과 및 고찰

본 실습을 통해 학사관리 업무의 핵심 엔티티와 관계를 관계형 모델로 설계하고 PostgreSQL에 구현하였다. 요구사항 정리부터 정규화 검토까지의 설계 절차를 거치며 PK·FK·UNIQUE·CHECK·ENUM으로 잘못된 데이터 입력을 구조적으로 제한하였고, 학생과 강좌의 N:M 관계를 수강신청 교차 테이블로 해소하여 학생별 수강내역과 강좌별 수강현황을 모두 조회할 수 있게 하였다.

조회 단계에서는 `WHERE`와 `ORDER BY`로 조건·정렬을 처리하고, `COALESCE`와 `CASE WHEN`으로 NULL 및 업무 문구(`미등록`, `미정`, 학년구분)를 변환하였다. 날짜 함수로 입학 연도와 경과 기간을 계산했으며, 다중 `JOIN`과 조건부 집계로 수강내역 및 잔여석을 확인하였다. 파생값을 저장하지 않고 조회 시 계산하도록 설계한 덕분에 원본 데이터와 표시 값의 불일치 없이 요구 정보를 얻을 수 있었다.